



REVAMP • LAUNCHPAD ⚡ V1.0

Set your laptop up like a developer.

A step-by-step guide for first-time builders.
Terminal, Git, Node, Python, Docker — and the
AI IDEs that turn your ideas into code.

MACOS • WINDOWS • LINUX

PRESS → TO BEGIN

WELCOME ABOARD.

By the end of this deck,
your laptop will be a
vibe-coding machine.

Zero experience required. We start with the terminal
and end with you shipping your first project.
Everything you install here is free.

WHAT YOU'LL INSTALL

Eight tools, three IDEs.

The stack any modern developer keeps on their machine. Two columns of essentials, one column of the AI editors that write code with you.

⚡ FREE • OPEN SOURCE • CROSS-PLATFORM

CLI & runtimes

Terminal • Package manager • Git & GitHub • SSH keys • Node.js • Python (uv) • Docker • Dotfiles

AI IDEs

Antigravity • Cursor • Gemini CLI • Claude Code + Windsurf, Copilot, Zed, VS Code

THE JOURNEY

Four legs to liftoff.

- | | | |
|----|-------------------------------------|---------|
| 01 | Pick your operating system | 2 MIN |
| 02 | Install the command-line essentials | 8 STEPS |
| 03 | Choose an AI IDE — and a backup | 3 PICKS |
| 04 | Ship your first vibe-coded project | TONIGHT |

PICK YOUR TRACK

Three OSes, same destination.

macOS

APPLE SILICON /
INTEL

12 (Monterey) or later. The default for most developers — Unix tools work out of the box.

```
$ uname -s  
Darwin
```

Windows

10 / 11

We'll install WSL2 (Linux inside Windows) so every command in this deck just works.

```
> wsl --version  
WSL version: 2.x
```

Linux

UBUNTU / DEBIAN
/ FEDORA

You're already home. Most commands assume apt; dnf swaps work fine.

```
$ uname -s  
Linux
```


The command line.

Eight tools every developer installs. Once. Then never thinks about again.

STEP 01 • TERMINAL

Your new home screen.

A text window where you talk to your computer. You'll live here.

macOS

WARP

```
brew install --cask warp
```

Modern, AI-native terminal. Or stick with the built-in [Terminal.app](#) / [iTerm2](#).

Windows

WINDOWS
TERMINAL + WSL

```
winget install  
Microsoft.WindowsTerminal  
wsl --install
```

WSL gives you real Ubuntu inside Windows. Reboot, then open Ubuntu from the Start menu.

Linux

GNOME / KONSOLE /
WARP

```
curl  
https://app.warp.dev/get_warp  
.sh | sh
```

Your distro ships with one already. [Warp](#) is a great upgrade if you want AI built in.

STEP 02 • PACKAGE MANAGER

One command, any app.

An app store for the terminal.
Installs and updates everything
else.

macOS

HOMEBREW

```
/bin/bash -c "$(curl -fsSL \
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Then `brew install <anything>`. The single most important tool on macOS.

Windows

WINGET • SCOOP

```
winget --version
# inside WSL Ubuntu:
sudo apt update && sudo apt
upgrade
```

`winget` ships with Windows 11. Inside WSL you'll use `apt`.

Linux

APT • DNF • PACMAN

```
sudo apt update
sudo apt install build-
essential
```

Already installed. `build-essential` gives you the C compilers other tools assume.

STEP 03 • GIT + GITHUB

Save points for code.

Time machine for your project.
Plus a public folder where the
world can see it.

macOS

BREW + GH

```
brew install git gh  
gh auth login
```

`gh` is GitHub's CLI. `auth login` connects your account once and forgets about it.

Windows

WINGET + WSL

```
winget install Git.Git  
GitHub.cli  
gh auth login
```

Run inside WSL Ubuntu so paths and line endings behave. Pick HTTPS when asked.

Linux

APT

```
sudo apt install git gh  
gh auth login
```

Set your name and email once: `git config --global user.name "..."`.

STEP 04 • SSH KEYS

A passwordless handshake.

Generate once. Push to GitHub forever, no password prompts.

●●● ~ — generate ssh key

```
# 1. Make a key (any OS, same command)
$ ssh-keygen -t ed25519 -C "you@example.com"
Generating public/private ed25519 key pair...
Enter file in which to save the key
(~/.ssh/id_ed25519):
# 2. Copy the public half
$ cat ~/.ssh/id_ed25519.pub | pbcopy
# Linux: xclip -sel c Win: clip.exe
# 3. Paste it into GitHub → Settings → SSH keys
$ ssh -T git@github.com
Hi yourname! You've successfully authenticated.
```

Three commands. [Make](#) → [Copy](#) → [Paste](#).

Git uses this key as your ID every time you push. No more typing passwords; no more 2FA prompts in the middle of work.

Pro move: also drop your key into `~/.ssh/config` so GitHub uses it automatically.

JavaScript for everything.

Powers most of the web — and almost every AI tool you'll install.

macOS

FNM + PNPM

```
brew install fnm pnpm
fnm install --lts
fnm use lts-latest
```

fnm swaps Node versions per project.
pnpm is faster than npm and saves disk.

Windows

INSIDE WSL

```
curl -fsSL
https://fnm.vercel.app/install | bash
fnm install --lts
npm i -g pnpm
```

Run from your WSL Ubuntu shell. Avoid installing Node on Windows directly — too many path bugs.

Linux

FNM + PNPM

```
curl -fsSL
https://fnm.vercel.app/install | bash
fnm install --lts
npm i -g pnpm
```

Re-source your shell or open a new tab so **fnm** takes effect.

STEP 06 • PYTHON + UV

The lingua franca of AI.

Use **uv** — the fast Python tool that replaces pip, venv, and pyenv.

macOS

UV VIA BREW

```
brew install uv
uv python install 3.12
uv init my-app && cd my-app
```

Forget about **pip**, **virtualenv**, and **pyenv**. **uv** does all three, 10× faster.

Windows

UV VIA
POWERSHELL

```
powershell -c "irm
https://astral.sh/uv/install.
ps1 | iex"
uv python install 3.12
```

Or inside WSL: **curl -LsSf**
https://astral.sh/uv/install.sh |
sh

Linux

UV VIA CURL

```
curl -LsSf
https://astral.sh/uv/install.
sh | sh
uv python install 3.12
```

Skip your distro's Python and let **uv** manage versions cleanly.

STEP 07 • DOCKER

Apps in tiny boxes.

Run any database, service, or someone else's project — without polluting your machine.

macOS

DOCKER DESKTOP

```
brew install --cask docker
open -a Docker
docker run hello-world
```

Docker Desktop bundles the engine + a GUI. Launch it once so the daemon starts.

Windows

DOCKER DESKTOP
+ WSL

```
winget install
Docker.DockerDesktop
# enable WSL2 integration in
settings
```

In settings, enable WSL integration for your Ubuntu distro so **docker** works in your shell.

Linux

DOCKER ENGINE

```
curl -fsSL
https://get.docker.com | sh
sudo usermod -aG docker $USER
```

Log out and back in so the group change sticks. Then **docker run hello-world**.

STEP 08 • SHELL + DOTFILES

Make the prompt yours.

A nicer shell, a colorful prompt,
and your settings on GitHub.

● ● ● ~ — make it cozy

```
# 1. Modern shell (default on mac, opt-in elsewhere)
$ chsh -s $(which zsh)
# 2. Pretty prompt that just works
$ brew install starship
$ echo 'eval "$(starship init zsh)"' >> ~/.zshrc
# 3. Track your config in a 'dotfiles' repo
$ mkdir ~/dotfiles && cd ~/dotfiles
$ git init && gh repo create --private
✓ Now your setup follows you to every machine.
```

Your shell is a [canvas](#).

Aliases, environment variables, AI keys, prompt theme, editor config — all live in plain text files in your home folder.

Push them to a private GitHub repo and you can re-set-up any laptop in 10 minutes.

EIGHT TOOLS, INSTALLED.

You just built a developer machine.



Terminal

WARP · ITERM2 · WINDOWS TERMINAL



Package manager

BREW · WINGET · APT



Git + GitHub CLI

GIT · GH AUTH LOGIN



SSH key on GitHub

SSH-KEYGEN -T ED25519



Node.js + pnpm

FNM INSTALL --LTS



Python + uv

UV PYTHON INSTALL 3.12



Docker

DOCKER RUN HELLO-WORLD



Shell + dotfiles

STARSHIP · ZSH · GIT REPO

The AI that writes code with you.

Three editors do 90% of the job. We'll cover all three —
then mention the rest.

THE THREE YOU ACTUALLY NEED

Pick one. Or use all three.

⚡ #1 • AGENT-FIRST
IDE

Antigravity

Google's free AI IDE. Multiple agents work in parallel, with browser-in-the-loop and verifiable artifacts.

Free

Models Gemini • Claude • GPT-
OSS

⚡ #2 • DAILY DRIVER
IDE

Cursor

The most popular AI IDE. Best inline autocomplete, mature Composer mode, huge community.

Price Free • \$20/mo Pro

Best for Daily editing

⚡ #3 • TERMINAL AGENTS

Gemini CLI / Claude Code

Gemini CLI — Google's free, open-source terminal agent. **Claude Code** — Anthropic's paid agent, best reasoning + 1M context. Pick by budget.

Gemini Free • OSS Claude \$20/mo+

ANTIGRAVITY • BY GOOGLE

Stop typing. Start **orchestrating**.

Antigravity treats the AI as the primary worker — you set the goal, agents handle the editor, the terminal, and a real Chrome browser. Free. Cross-platform.

Editor view — familiar VS Code feel with an agent sidebar.

Manager view — mission control for many agents at once.

Artifacts — agents return screenshots, plans, and recordings, not raw tool calls.

```
●●● install · all OS
```

```
# Download from antigravity.google  
# macOS  
$ brew install --cask antigravity  
# Windows  
> winget install Google.Antigravity  
# Linux — .deb / .rpm from the site  
$ sudo dpkg -i antigravity*.deb
```

Sign in with Google. Pick **Gemini 3 Pro** for hard tasks, **Flash** for speed.

CURSOR • THE DAILY DRIVER

Tab. Tab. Ship.

A VS Code fork with AI baked into every keystroke. The autocomplete feels telepathic; Composer rewrites multiple files from a single prompt.

Tab — predictive autocomplete that learns your patterns.

Composer — multi-file edits from natural language.

Agent — fully autonomous task execution.

```
●●● install • all OS
```

```
# macOS
$ brew install --cask cursor
# Windows
> winget install Anysphere.Cursor
# Linux — .AppImage from cursor.com
$ chmod +x cursor*.AppImage &&
./cursor*.AppImage
✓ Drop your VS Code settings in on first
launch.
```

Free tier is real. Pro at \$20/mo unlocks heavy agent usage and Claude Sonnet / GPT-5 / Gemini 3 by toggle.

TERMINAL AGENTS · NO EDITOR REQUIRED

Just a **brain** in your terminal.

FREE PATH · GOOGLE

Gemini CLI

 gemini · all OS

```
# Install
$ npm i -g @google/gemini-cli
$ cd my-project && gemini
> add magic-link auth
✓ Edited 4 files, ran tests.
```

Free · OSS · Sign in with Google.

PAID PATH · ANTHROPIC

Claude Code

 claude · all OS

```
# Install
$ npm i -g @anthropic-ai/claude-code
$ cd my-project && claude
> refactor auth module
✓ Spawned 3 sub-agents.
```

\$20/mo+ · Best reasoning · 1M context.

WORTH KNOWING

The rest of the field.

PLUGIN • VS CODE

GitHub Copilot

The original. Best if you live in VS Code or JetBrains, and especially good for teams already on GitHub.

From \$10/mo

IDE • PIONEERED
CASCADE

Windsurf

Cheapest paid tier. Cascade agent + persistent context — the AI watches what you do and jumps in unprompted.

From Free • \$15/mo

SPEED-FIRST
EDITOR

Zed

Built in Rust. Materially lighter than Electron editors. Real-time collaboration baked in.

Free

THE CLASSIC

VS Code

Still the most popular code editor. Add Copilot or Cline as extensions for AI; everything else is here too.

Free • OSS

RECOMMENDED COMBOS

Pick a stack. Don't overthink it.

⚡ FREE FOREVER

Antigravity solo

Free Google IDE with Gemini 3, three model options, multi-agent orchestration. Zero monthly cost.

\$0/mo

⚡ MOST POPULAR

Cursor + Gemini CLI

Cursor for typing, Gemini CLI in a terminal tab for long, complex tasks. Free terminal agent + \$20/mo IDE.

~\$20/mo

⚡ POWER USER

All three at once

Antigravity for parallel agents, Cursor for daily flow, Claude Code for the hardest tickets. Overkill, in a good way.

~\$40-120/mo

YOUR FIRST VIBE-CODED PROJECT

Five lines. One running app.

●●● ~ — first launch

```
# 1. New folder, new git repo
$ mkdir hello-world && cd hello-world && git init
# 2. Open it in your AI IDE of choice
$ cursor . # or: antigravity .
# 3. Tell it what you want
> "build me a Pomodoro timer as a single HTML page"
# 4. Push to GitHub from the terminal
$ gh repo create --public --push --source=.
✓ https://github.com/you/hello-world
```

That's it. You're a developer.

No bootcamp. No tutorial hell. The fastest path from "I have an idea" to "it's running on the internet" is the one you just walked.

Next up: deploy with [vercel](#), add a database with [supabase](#), and ship.

PRE-FLIGHT

Ready for liftoff?

**Terminal opens without errors**`ECHO HELLO`**git config name + email set**`GIT CONFIG --GLOBAL USER.NAME`**node + pnpm work**`NODE -V · PNPM -V`**docker hello-world prints**`DOCKER RUN HELLO-WORLD`**Package manager installs apps**`BREW · WINGET · APT`**SSH key on GitHub**`SSH -T GIT@GITHUB.COM`**uv runs python**`UV RUN PYTHON --VERSION`**An AI IDE is open**`ANTIGRAVITY · CURSOR · GEMINI CLI / CLAUDE
CODE`

BOOKMARK THESE

When you get stuck.

CORE

[brew.sh](#)[git-scm.com/docs](#)[cli.github.com](#)[docs.astral.sh/uv](#)

RUNTIMES

[nodejs.org](#)[pnpm.io](#)[docs.docker.com](#)[starship.rs](#)

AI IDEs

[antigravity.google](#)[cursor.com](#)[github.com/google/gemini-cli](#)[claude.com/code](#)

WHEN STUCK

[stackoverflow.com](#)[claude.ai](#)[chatgpt.com](#)[your AI IDE chat](#)

⚡ LIFTOFF

Now go **build** something.

You have the tools, the IDE, and the AI.
The only thing left is the idea.

